

Cẩm nang vận hành đội AI từ xa

Hướng dẫn chi tiết claude-telegram-multi-relay

hailoc12

06/2026

Mục lục

Lời giới thiệu

Bạn cài Claude Code. Nó giỏi thật: đọc file, sửa code, chạy lệnh, deploy. Nhưng tất cả diễn ra trong một cửa sổ terminal, và bạn phải ngồi trước máy mới dùng được.

Cuốn sách này giải một bài toán hẹp nhưng thực tế: điều khiển nhiều Claude Code cùng lúc, từ điện thoại, bất kể bạn đang ở đâu, và để chúng tự chạy theo lịch, tự gỡ rối khi hỏng.

Công cụ là `claude-telegram-multi-relay`, một lớp điều phối nằm trên ccc (Claude Code Companion của tác giả kidandcat). Cách dùng nó: mỗi Claude trở thành một “nhân viên số” có riêng một kênh chat trên Telegram. Bạn giao việc, nhận báo cáo, lên lịch ca trực. Cũng giống cách quản lý một đội người thật, chỉ là đội số.

Sách viết cho ba kiểu người:

- Đã dùng Claude Code, muốn mở rộng từ một terminal sang một đội.
- Solopreneur hoặc chủ doanh nghiệp nhỏ đang dựng AI Native Company, nơi một người vận hành nhiều AI Worker.
- Kỹ sư muốn tự động hóa công việc nhàm chán qua Telegram và lịch trình.

Đọc xong, bạn tự dựng được một đội Claude nhiều phiên, mỗi phiên một topic Telegram, có lịch tự chạy và tự chữa.

Lưu ý nền tảng: Sách viết cho macOS, vì phần lập lịch dựa vào `launchd`. Phần relay lỗi chạy đa nền tảng, nhưng bản hoàn chỉnh cần macOS.

Mục lục

1. Telegram Relay là gì
2. Kiến trúc hệ thống
3. Chuẩn bị môi trường và tạo bot Telegram
4. Cài đặt và cấu hình
5. Quản lý phiên (session)
6. Đa luồng: multi-topic, multi-user, multi-group
7. Phối hợp đội (broadcast)

8. Lập lịch định kỳ với launchd
9. Giám sát, tự phục hồi và kiểm toán
10. Tích hợp với AI Native Company
 - Phụ lục A: Bảng quy tắc R-00 đến R-27
 - Phụ lục B: Khắc phục sự cố
 - Phụ lục C: Thuật ngữ

Chương 1: Telegram Relay là gì

Vấn đề của Claude Code

Claude Code là CLI. Mở terminal, gõ lệnh, đọc kết quả ngay trên màn hình đó. Tuyệt khi bạn đang ngồi trước máy. Sụp đổ trong bốn tình huống:

- Bạn đang ngoài đường, chỉ có điện thoại. Muốn Claude kiểm tra một log lỗi, nhưng máy để ở nhà.
- Có mười việc cần chạy song song: nghiên cứu mười đối thủ, theo dõi mười luồng CI. Mỗi việc cần một Claude riêng. Không ai mở mười terminal rồi canh từng cái.
- Một việc chạy ba mươi phút. Bạn không muốn ngồi chờ, mà muốn nó báo qua điện thoại khi xong, rồi trả lời tiếp ngay trên điện thoại.
- Bạn muốn Claude **tự chạy mỗi sáng tám giờ** để tổng hợp báo cáo, không cần mở máy lên.

Tất cả quy về một nhu cầu: một kênh liên lạc giữa bạn và Claude, thoát khỏi chiếc terminal.

Relay là trạm trung chuyển

Relay là phần mềm đứng giữa Claude Code và Telegram:

bạn (điện thoại) <--> Telegram <--> RELAY <--> Claude Code

Nó làm hai việc, không hơn.

Thứ nhất, hướng vào: bạn gõ tin trong Telegram, relay đọc được, đẩy vào Claude đang chạy. Thứ hai, hướng ra: Claude xuất kết quả, relay bắt lấy, gửi về Telegram.

Hai chiều độc lập, chạy thời gian thực. Bạn và Claude trò chuyện qua Telegram như chat với đồng nghiệp, nhưng “đồng nghiệp” đó đọc và sửa file, chạy lệnh, và triển khai dịch vụ ngay trên máy bạn.

💡 **MẸO:** Hãy nghĩ relay như một trợ lý đứng cạnh máy bạn. Bạn nhắn cho trợ lý qua Telegram, trợ lý đọc to cho Claude, rồi chép câu trả lời gửi lại bạn. Relay tự động hóa đúng vai trò đó.

Vì sao chọn Telegram

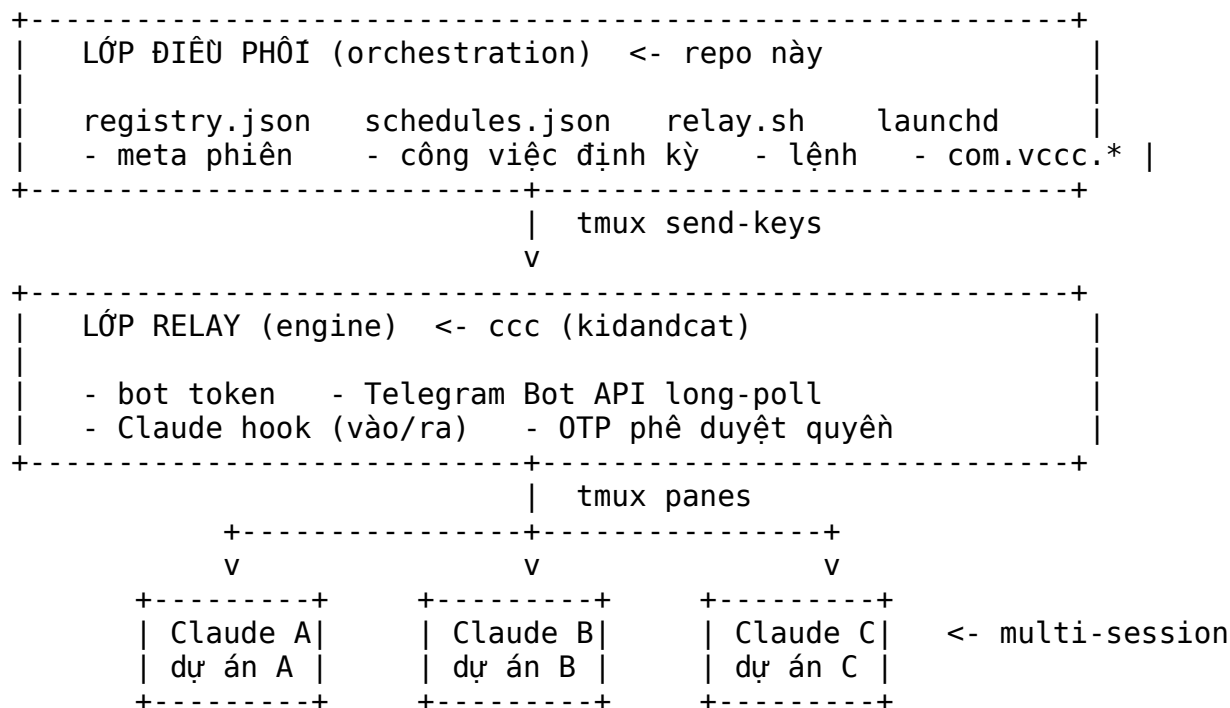
Ứng dụng nhắn tin thì nhiều. Telegram được chọn vì bốn lý do thực dụng. Có sẵn trên mọi thiết bị, từ điện thoại đến trình duyệt. Bot API mạnh và miễn phí: tạo chủ đề, gửi file, nút bấm, phát

Một lý do tinh tế hơn: Telegram không rate-limit khắt k như email hay SMS, hợp với việc Claude hay gửi nhiều tin nhỏ liên tục.

Chương 2: Kiến trúc hệ thống

Hai lớp tách biệt

Hiệu kiến trúc là chìa để dùng và gỡ rời. Hệ thống gồm hai lớp xếp chồng:



Một nguyên tắc cần nắm lòng: repo này KHÔNG viết lại relay. Nó chỉ đọc và ghi cấu hình của ccc (nguồn dữ liệu chính cho kết nối), rồi thêm ba thứ ccc chưa có: registry (siêu dữ liệu điều phối), schedules (công việc định kỳ), scripts (vòng đời phiên).

ccc, phần nền móng

ccc (Claude Code Companion) là engine relay do kidandcat phát triển, mã nguồn mở tại github.com/kidandcat/ccc. Nó lo phần nền: quản lý bot token, lắng nghe Telegram Bot API, cài hook vào Claude để bắt tin vào ra, phê duyệt quyền qua OTP (khi Claude xin chạy lệnh nhạy cảm, ban duyệt ngay trên Telegram), chuyển tiếp file và các lệnh `/new`, `/continue`, `/c`.

Phần này ổn định, qua thử nghiệm thực tế. Repo của chúng ta kế thừa, không làm lại.

Lớp điều phối, phần repo này đóng góp

Bản thân ccc xử lý tốt **một phiên ứng với một topic**. Khi bạn muốn nhiều phiên chạy song song, tự lên lịch, tự phục hồi, ccc chưa có cơ chế. Khoảng trống đó repo này lấp:

Đóng góp	Mô tả
Điều phối multi-session	registry + vòng đời (tạo/gửi/list/trạng thái/xóa)
Multi-topic	tự tạo topic Telegram, map một-một với mỗi phiên
Multi-user / multi-group	một bot phục vụ nhiều nhóm, mỗi phiên nhớ <code>group_id</code>
Lập lịch bền vững	biểu thức cron sang launchd, sống qua khởi động lại và thức giấc
Quản lý đội	giám sát, broadcast, tự khởi động lại, kiểm toán

Bốn nguồn dữ liệu phải đồng bộ

Có bốn nơi lưu trạng thái. Thay đổi gì thì phải cập nhật đủ, không sẽ lệch (rule R-12):

Nguồn	Vai trò
<code>ccc config.json</code>	Kết nối: token, group_id, danh sách phiên (topic/path/window)
<code>registry.json</code>	Siêu dữ liệu điều phối: mục đích, thẻ, trạng thái, tự khởi động
<code>schedules.json</code>	Công việc định kỳ: biểu thức cron, prompt, lịch sử chạy
<code>com.vccc.*.plist</code>	Định nghĩa tác vụ launchd

Lệnh `relay_cron_list` tự phát hiện “mồ hôi”: plist tồn tại mà không có entry trong registry, hoặc ngược lại, để bạn dọn.

⚠ **LƯU Ý:** Đừng sửa tay bốn file này tùy tiện. Luôn dùng lệnh `relay_*`, chúng đảm bảo đồng bộ. Sửa tay là nguyên nhân số một gây lệch trạng thái.

Chương 3: Chuẩn bị môi trường và tạo bot Telegram

Kiểm tra điều kiện

Trước khi cài, đảm bảo máy có đủ công cụ. Mở terminal, chạy từng lệnh kiểm tra:

```
claude --version    # Claude Code CLI
ccc --version        # relay engine (kidandcat/ccc)
```

```
tmux -V          # qua'n lý cu'a sô' terminal
python3 --version # ngôn ngữ kịch ba'n
```

Thiếu cái nào thì cài:

```
npm i -g @anthropic-ai/claude-code # Claude Code
brew install tmux python           # tmux + python3
```

ccc cài theo hướng dẫn của kidandcat tại github.com/kidandcat/ccc. Đây là phụ thuộc bắt buộc, repo này điều phối ccc chứ không thay thế nó.

💡 **MẸO:** Chạy ccc doctor ngay sau khi cài. Lệnh kiểm tra toàn bộ phụ thuộc và cấu hình, báo rõ cái nào thiếu. Khắc phục hết rồi mới đi tiếp.

Tạo bot Telegram

Bot là danh tính của hệ thống trên Telegram. Tạo một lần thôi.

Một. Mở Telegram, tìm **@BotFather**. Hai. Gõ /newbot, đặt tên (ví dụ “Đội AI của tôi”) và username phải kết thúc bằng bot (ví dụ my_ai_team_bot). Ba, BotFather trả về token API dạng 123456789:ABCdef... Đây là chìa khóa điều khiển bot, giữ kín, đừng đẩy lên Git.

123456789:AAH-xyz...ABCdef <- token, đáng thê' này

Tạo nhóm có Topics

Tiếp theo là không gian làm việc: một nhóm bật Forum Topics.

Tạo nhóm mới, thêm bot vừa tạo vào làm quản trị viên. Vào cài đặt, bật Topics (cần nhóm dạng supergroup). Giờ nhóm thành forum, mỗi phiên Claude sẽ tự là một topic.

Lấy group_id (số định danh nhóm) bằng cách thêm tạm @getidsbot hoặc @RawDataBot vào nhóm. Nó báo một số dạng -100xxxxxxxxxx. Ghi lại, rồi xóa bot đó đi.

⚠️ **LƯU Ý:** group_id luôn âm và bắt đầu bằng -100 vì đó là supergroup. Đúng rồi, đừng tự bỏ dấu trừ.

Cấu hình token vào ccc

Token không nằm trong repo này, mà nằm trong cấu hình của ccc. Cấu hình một lần:

```
ccc setup <DÁN_TOKEN_TỪ_BOTFATHER_VÀO_ĐÂY>
```

ccc setup làm ba việc: xác thực token, cài hook vào Claude, đăng ký dịch vụ launchd để lắng nghe Telegram. Xong thì cấu hình nhóm:

```
ccc setgroup      # nhập group_id đê' /new tạo topic hoạt động
ccc doctor        # kiê'm tra lầ'n cuô'i
```

Kiểm tra cấu hình đã nhận:

```
cat ~/.config/ccc/config.json | python3 -m json.tool
```

Phải thấy bot_token, group_id, chat_id. Đủ thì nền móng xong.

Chương 4: Cài đặt và cấu hình

Tải repo về

```
git clone https://github.com/hailoc12/claude-telegram-multi-relay.git
cd claude-telegram-multi-relay
```

Cấu trúc thư mục:

```
claude-telegram-multi-relay/
├── SKILL.md                # Skill Claude Code (logic + quy tắc)
├── scripts/
│   ├── relay.sh            # thư viện lỗi: tạo/gửi/broadcast/cron
│   ├── run-scheduled.sh   # cầu nối launchd <-> Claude
│   └── cron_to_launchd.py  # chuyển biểu thức cron sang plist
├── config/                 # ví dụ cấu hình
├── templates/              # ví dụ launchd agent
└── docs/                   # tài liệu chuyên sâu
```

Cài Skill vào Claude Code

Để Claude hiểu các lệnh điều phối, cài SKILL.md vào nơi nó đọc skill:

```
SKILL_DIR=~/.claude/skills/claude-telegram-multi-relay
mkdir -p "$SKILL_DIR"
cp SKILL.md scripts -t "$SKILL_DIR"
```

Từ giờ, trong một phiên Claude Code, bạn nói tự nhiên được: “tạo ba phiên nghiên cứu đối thủ”, hoặc “lên lịch báo cáo mỗi sáng tám giờ”.

Cài đặt cầu nối lập lịch

Phần lập lịch cần một thư mục dữ liệu và script cầu nối:

```
ORCH=~/.claude/ccc-orchestrator
mkdir -p "$ORCH/logs"
cp scripts/run-scheduled.sh "$ORCH/"
cp config/relay.example.env "$ORCH/relay.env"
```

Tùy chọn cấu hình

File relay.env cho phép ghi đè đường dẫn nếu máy bạn khác mặc định. Hầu hết người dùng không cần đụng tới. Chỉ bỏ dấu thăng (#) và sửa khi binary CCC không nằm trong PATH (đặt CCC_BIN), hoặc thư mục dự án gốc khác mặc định (đặt PROJECTS_DIR):

```
# Ví dụ ghi đè:
#CCC_BIN="/usr/local/bin/ccc"
#PROJECTS_DIR="$HOME/Projects"
```

Mặc định PROJECTS_DIR trỏ tới thư mục iCloud Documents/9. active, đồng bộ qua thiết bị. Dùng thư mục khác thì đặt cho đúng.

Kiểm tra

Chạy kiểm tra cuối cùng:

```
source scripts/relay.sh
relay_list # chưa có phiên nào cũng đúng, chi' cần không báo lỗi
```

Lệnh chạy mà không lỗi phụ thuộc, là cài đặt thành công.

 **BÀI TẬP:** Tạo phiên đầu tiên ngay để kiểm tra toàn bộ đường truyền. Chạy `relay_new test-session "Kiểm tra kết nối"`, rồi vào nhóm Telegram xem bot có tự tạo topic và gửi thông báo sẵn sàng không. Thấy là toàn bộ đường ống đã thông.

Chương 5: Quản lý phiên (session)

Phiên là đơn vị cơ bản: một Claude Code chạy trong một cửa sổ tmux, gắn với một topic Telegram và một thư mục dự án. Chương này đi qua vòng đời phiên.

Tạo phiên mới

```
source scripts/relay.sh
relay_new ten-session "Mục đích của phiên"
```

Chạy lệnh này, hệ thống tự làm chín bước: kiểm tra tên hợp lệ (chữ thường, gạch ngang, chưa tồn tại); tạo thư mục dự án kèm `.claude/settings.json` để bỏ qua prompt quyền; tạo topic Telegram qua API, nhận `topic_id`; mở cửa sổ tmux mới; khởi động Claude qua `ccc run`; đổi tên cửa sổ tmux về đúng tên phiên; ghi vào cấu hình `ccc` (topic, đường dẫn, cửa sổ); ghi vào registry (mục đích, trạng thái active); gửi thông báo sẵn sàng vào topic.

Bạn sẽ thấy trên Telegram:

✅ Session ten-session ready
Purpose: Mục đích của phiên
Dir: /path/to/project
Window: @19

Và trong nhóm xuất hiện một topic mới tên `ten-session`. Mọi tin nhắn bạn viết trong topic đó đi thẳng vào Claude.

💡 **MẸO:** Quy tắc đặt tên: chữ thường, gạch ngang, ngắn. Tên tốt: `comp-research-a`, `daily-report`, `ci-monitor`. Tên xấu: `My Session!`, có dấu tiếng Việt hoặc ký tự lạ.

Gửi lệnh tới phiên

Có hai cách. Tự nhiên nhất: gõ trực tiếp trong topic Telegram, Claude nhận ngay. Cách thứ hai, từ terminal:

```
relay_send ten-session "Phân tích điểm yếu đối thủ A, lưu ra report.md"
```

Hệ thống kiểm tra phiên còn sống, đẩy prompt vào cửa sổ tmux, cập nhật mốc thời gian lệnh cuối. Cửa sổ đã chết thì xử lý theo cài đặt tự khởi động lại.

Liệt kê toàn bộ phiên

```
relay_list
```

Kết quả dạng bảng, cho bạn bức tranh cả đội:

```
📖 Sessions (4 registered)
```

comp-research-a	● active	win:@19	topic:785	Nghiên cứu đối thủ A
comp-research-b	● active	win:@20	topic:786	Nghiên cứu đối thủ B
daily-report	● idle	win:@21	topic:787	Báo cáo hàng ngày
dead-session	● dead	win:@22	topic:788	Phiên đã tắt

Trạng thái được suy ra từ việc cửa sổ tmux có còn tồn tại: xanh khi đang chạy, vàng khi nhàn rỗi, đỏ khi tắt, xám khi chưa theo dõi.

Kiểm tra sức khỏe

```
relay_status ten-session
```

Lệnh kiểm tra ba lớp: cửa sổ tmux còn không, tiến trình Claude có chạy trong pane không, nội dung pane có dấu lỗi không. Trả về trạng thái kèm mã thoát, có ích khi viết script tự động.

Xóa phiên

```
relay_kill ten-session
```

Hệ thống gửi Ctrl-C rồi /exit, đợi, ép đóng cửa sổ nếu còn, dọn cấu hình ccc, đánh dấu registry là **terminated**. Qua skill, bước này hỏi xác nhận bằng nút bấm trên Telegram (rule R-01), tránh xóa nhầm.

Chuyển thư mục dự án

Thao tác tinh tế: bạn muốn đổi thư mục làm việc của một phiên, nhưng **giữ nguyên topic**, để lịch sử chat và danh tính phiên không đổi.

Nói rõ: đây là thao tác **tạo lại (recreate)**, không phải chỉ cập nhật cấu hình (rule R-23). Trình tự bảy bước. Đọc thông tin cũ (cửa sổ, topic, đường dẫn). Đổi tên cửa sổ cũ thành `ten-session-old`, tránh nhầm khi tìm kiếm. Tạo thư mục mới kèm cấu hình bỏ quyền. Mở cửa sổ tmux mới, chạy `ccc run`. Cập nhật cấu hình ccc: đường dẫn và cửa sổ mới, giữ nguyên `topic_id`. Cập nhật registry. Đóng cửa sổ cũ, kiểm tra đã thực sự tắt. Cuối cùng, thông báo vào topic.

Sau bước này, bạn vẫn chat trong cùng topic, nhưng Claude giờ làm việc ở thư mục mới.

⚠ **LƯU Ý:** Sai lầm phổ biến, và là lý do rule R-23 tồn tại: chỉ cập nhật cấu hình mà quên tạo cửa sổ tmux mới, quên chạy `ccc run`, quên đóng phiên cũ. Kết quả là cấu hình nói thư mục mới, còn Claude vẫn chạy ở thư mục cũ. Phải làm đủ bảy bước.

Chương 6: Đa luồng: multi-topic, multi-user, multi-group

Đây là phần tạo nên khác biệt cốt lõi của repo. Relay đơn giản chỉ nối một Claude với một chat. Repo này thêm ba chiều “multi”.

Multi-session, nhiều Claude song song

Mỗi phiên là một Claude riêng: thư mục riêng, ngữ cảnh riêng, lịch sử riêng. Bạn chạy được mười phiên cùng lúc (giới hạn thực tế là RAM và CPU, thường mười đến hai mươi trên một máy thường).

Tạo nhiều phiên, mỗi cái tự có một topic Telegram. Mở nhóm forum, bạn thấy:

```
📁 Nhóm Làm Việc (forum)
├── ● ai_native_company
├── ● comp-research-a
├── ● comp-research-b
├── ● daily-report
└── ● dead-session
```

Click vào topic nào là đang nói chuyện với phiên đó. Tách bạch, rõ ràng.

Multi-topic, mỗi phiên một kênh

Telegram Forum Topics biến nhóm thành nhiều kênh (như Slack channels). Khi tạo phiên, hệ thống gọi API `createForumTopic`, nhận `topic_id`, map một-một phiên với topic.

Hướng ra cũng vậy: Claude trả lời, hook của `ccc` tra `topic_id` của phiên hiện tại, gửi vào đúng topic. Bạn không bao giờ bị lẫn phản hồi giữa các phiên.

Multi-user / multi-group, một bot nhiều không gian

Đây là chiều mạnh nhất. **Một bot token phục vụ nhiều nhóm khác nhau**, vì Telegram Bot API vốn hỗ trợ tự nhiên. Mỗi phiên lưu riêng `group_id`:

```
phiên "work-x"    -> group_id -1003460044206  (Nhóm Công việc)
phiên "family"    -> group_id -1003918647978  (Nhóm Gia đình)
```

Cơ chế routing rất gọn:

```
nhóm thực tế' = phiên.group_id  nếu có
               = config.group_id nếu phiên chưa có (fallback)
```

Ý nghĩa: một nhóm cho công ty A, một nhóm cho khách B, một nhóm cá nhân, tất cả dùng chung một bot nhưng mỗi không gian hoàn toàn tách. Đó chính là “multi-user”.

Thiết lập multi-group

Ba bước. Thêm bot vào nhiều nhóm, mỗi nhóm bật Topics và bot làm quản trị viên. Trong mỗi nhóm chạy `/new <tên>`, phiên tự gắn với nhóm đó. Xong. Bot phục vụ tất cả song song.

 **ĐIỂM CHÍNH:** Tương thích ngược: phiên cũ không có `group_id` tự dùng `group_id` mặc định ở mức cấu hình. Không cần di chuyển dữ liệu khi lên multi-group.

Chương 7: Phối hợp đội (broadcast)

Đội đông rồi, sẽ có lúc cần gửi cùng một lệnh cho nhiều phiên cùng lúc. Đó là broadcast.

Cú pháp

`relay_broadcast "prompt cần gửi" [mục tiêu]`

Mục tiêu có ba dạng: `all` cho tất cả phiên đang active; `group:tên` cho các phiên thuộc một nhóm logic; `tag:tên` cho các phiên mang một thẻ nào đó.

Ví dụ: đội nghiên cứu song song

Giả sử muốn nghiên cứu năm đối thủ cùng lúc:

```
for c in a b c d e; do
  relay_new "comp-$c" "Nghiên cứu đối thủ $c"
done
```

```
relay_broadcast \
  "Nghiên cứu sâu: vị thế thị trường, giá, tính năng, điểm yếu. Lưu ra
output/<tên>.md" \
  all
```

Năm Claude nhận cùng lệnh, làm song song, mỗi cái lưu kết quả vào thư mục riêng. Bạn theo dõi tiến độ qua năm topic trên Telegram.

Nhóm và thẻ

Gán phiên vào nhóm logic hoặc gắn thẻ, rồi broadcast theo nhóm:

```
# (Qua skill hoặc chỉ'nh registry.json)
# nhóm "research-team" chứa comp-a, comp-b, comp-c
```

```
relay_broadcast "Tổng hợp phát hiện chính, mỗi phiên 3 điểm"
group:research-team
```

Cách này tổ chức đội theo dự án hoặc theo vai trò.

Giới hạn và an toàn

Broadcast giới hạn mười phiên cùng lúc (rule R-07) để tránh quá tải. Mỗi phiên được gửi độc lập, lỗi ở phiên này không kéo theo phiên kia. Xong, bạn nhận báo cáo: “đã gửi N, thất bại M”.

⚠ **LƯU Ý:** Broadcast gửi prompt bằng nhau cho mọi phiên. Nếu các phiên có ngữ cảnh khác nhau nhiều, cần nhắc gửi riêng từng cái, hoặc dùng thẻ để nhóm các phiên tương đồng.

Chương 8: Lập lịch định kỳ với launchd

Đây là tính năng biến đội Claude từ “chạy khi được gọi” thành “tự chạy theo lịch”. Chương giải thích vì sao launchd chứ không cron, rồi cách vận hành.

Vì sao launchd, không cron

macOS có hai bộ lập lịch: cron (cũ) và launchd (native). Repo này mặc định launchd, vì bốn lý do.

Apple đã ngừng khuyến khích cron, launchd là bộ lập lịch gốc của macOS. Cron bỏ qua tác vụ khi Mac ngủ; launchd chạy khi máy thức dậy, quan trọng với máy để qua đêm. launchd sống qua khởi động lại, vì plist trong ~/Library/LaunchAgents tự nạp lại. Một plist bằng một tác vụ, nên dễ quản lý, sao lưu, chia sẻ, nhóm bằng tiền tố. Hơn nữa bản thân CCC đã chạy dưới dạng dịch vụ launchd, dùng cùng cơ chế cho nhất quán.

Cú pháp cron quen thuộc

Người dùng quen với biểu thức cron năm trường: phút giờ ngày tháng thứ. Bạn viết bằng cú pháp quen thuộc này, một script chuyển đổi (cron_to_launchd.py) tự dịch sang plist launchd nội bộ.

Vài mẫu phổ biến:

Biểu thức cron	Ý nghĩa	Chuyển sang launchd
7 9 * * 1-5	9h07 các ngày thường	StartCalendarInterval (5 ngày)
*/15 * * * *	mỗi 15 phút	StartInterval 900
13 * * * *	phút 13 mỗi giờ	StartCalendarInterval Minute=13
0 18 * * 5	18h00 thứ Sáu	StartCalendarInterval Hour=18,Weekday=5

Toàn vòng đời CRUD

Quản lý tác vụ định kỳ đầy đủ tạo-đọc-sửa-xóa:

```
# Tạo
relay_cron_create daily-standup "7 9 * * 1-5" daily-report \
```

"Tổng hợp standup hôm nay với số liệu hôm qua"

```
# Liệt kê (kiểm tra ca' trạng thái nạp launchd)  
relay_cron_list
```

```
# Tạm dừng (giữ plist trên đĩa)  
relay_cron_toggle daily-standup false
```

```
# Bật lại  
relay_cron_toggle daily-standup true
```

```
# Xem nhật ký (launchd tự ghi stdout/stderr)  
relay_cron_history daily-standup
```

```
# Xóa hã'n  
relay_cron_delete daily-standup
```

Quy ước đặt tên

Mọi tác vụ launchd dùng nhãn `com.vccc.<tên>` (rule R-05). Nhờ tiền tố thống nhất, liệt kê cả đội bằng:

```
launchctl list | grep com.vccc
```

Tên tác vụ không chứa dấu chấm (rule R-17), để tương thích định dạng nhãn.

Cầu nối run-scheduled.sh

Khi launchd kích hoạt, nó gọi `run-scheduled.sh <tên-tác-vụ>`. Script này làm: đặt PATH đầy đủ (launchd có PATH tối thiểu, sẽ thiếu tmux và python nếu không đặt, rule R-19); kiểm tra phụ thuộc (tmux, python3, ccc) trước khi chạy, báo lỗi rõ thay vì sụp đổ mơ hồ (rule R-21); đọc định nghĩa tác vụ từ `schedules.json`; kiểm tra phiên mục tiêu còn sống, nếu chết và bật auto-create thì tự tạo lại phiên rồi mới gửi prompt; đẩy prompt vào cửa sổ tmux; cập nhật mốc chạy cuối và trạng thái; gửi thông báo vào topic Telegram.

💡 **MẸO:** Mọi hàm trong script cầu nối phải định nghĩa **trước** lần gọi đầu tiên (rule R-20). Bash không “nâng” hàm như nhiều ngôn ngữ: định nghĩa hàm ở cuối mà gọi ở giữa sẽ sụp đổ âm thầm. Đây là một trong những bug khó bắt nhất.

Chương 9: Giám sát, tự phục hồi và kiểm toán

Đội đông thì cần giám sát. Chương này lo việc: ai đang sống, ai đã chết, ai cần cứu.

Giám sát toàn đội

Chạy kiểm tra sức khỏe tất cả phiên:

```
for s in $(relay_list --names); do relay_status "$s"; done
```

Hoặc lên lịch tự giám sát mỗi ba mươi phút:

```
relay_cron_create fleet-monitor "*/30 * * * *" monitor \  
"Kiểm tra sức khỏe toàn bộ phiên"
```

launchd sẽ đánh thức Mac nếu đang ngủ để chạy việc này, điều cron không làm được.

Tự phục hồi

Mỗi phiên trong registry có cờ `auto_restart`. Khi phiên chết và cờ đang bật, hệ thống tự khởi động lại: tạo cửa sổ tmux, chạy `ccc run`, cập nhật cấu hình. Có giới hạn ba lần mỗi giờ (rule R-09), chết quá nhiều thì báo con người thay vì vòng lặp khởi động vô tận.

Cầu nối lập lịch cũng self-heal riêng: phiên mục tiêu đã chết mà tác vụ bật `auto_create_session` thì tự tạo phiên rồi mới gửi prompt. Tác vụ định kỳ không bao giờ “bom” chỉ vì phiên tình cờ tắt.

Kiểm toán

Sau mỗi lần tạo hoặc sửa tác vụ, nên chạy kiểm toán (rule R-22). Kiểm toán quét cả đội và báo: script cầu nối có đặt PATH đầy đủ không (R-19); hàm định nghĩa trước lần gọi chưa (R-20); có kiểm tra phụ thuộc đầu không (R-21); mỗi tác vụ có plist đã nạp vào launchd chưa; phiên mục tiêu còn sống hoặc sẽ tự tạo; `last_status` có lỗi không; phát hiện “mồ côi”: plist mà không có entry registry, và ngược lại.

Kết quả dạng số: “ N đạt,  M cảnh báo,  K thất bại”, kèm danh sách cần sửa.

Nhật ký

launchd tự ghi stdout vào `.log` và stderr vào `.err` qua `StandardOutPath` trong plist (rule R-14), nên bạn không cần tự tee. Tác vụ lỗi thì xem:

```
relay_cron_history daily-standup 30
```

Hiển thị ba mươi dòng cuối của cả stdout và stderr.

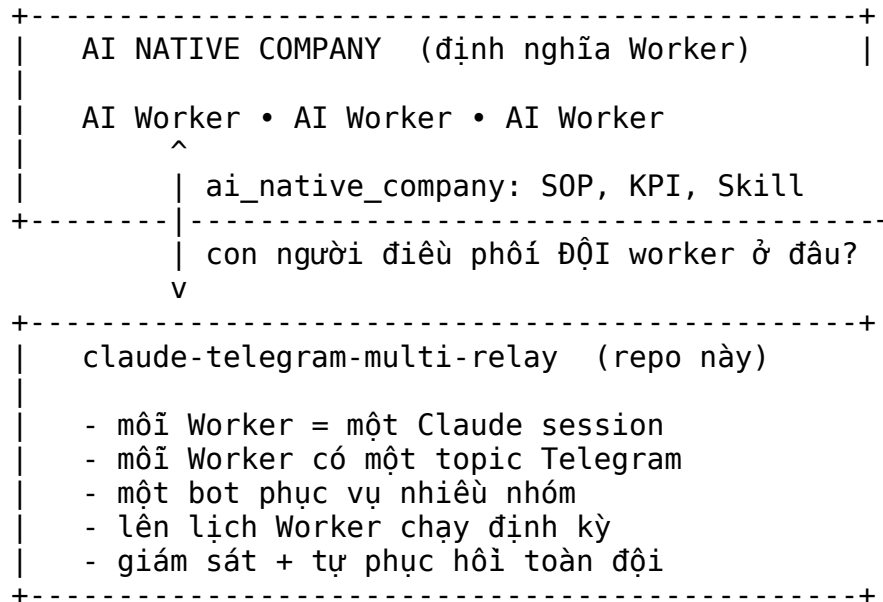
 **ĐIỂM CHÍNH:** Tự phục hồi cộng lập lịch bền vững cộng kiểm toán tạo nên một đội “tự quản”. Bạn đi cả tuần, đội vẫn chạy đúng lịch, tự sửa khi sự cố, và bạn chỉ cần đọc báo cáo trên điện thoại.

Chương 10: Tích hợp với AI Native Company

Repo này không đứng đơn lẻ. Nó là **tầng giao tiếp và vận hành của con người** trong mô hình AI Native Company (github.com/hailoc12/ai_native_company).

Vị trí trong mô hình

AI Native Company định nghĩa **AI Worker**: nhân sự số chuyên biệt, mỗi worker có vai trò, quy trình (SOP), chỉ số chất lượng (SLI/KPI), kỹ năng và kiến thức. Một người vận hành với năng lực tương đương mười lăm đến hai mươi người.



Nói cách khác: `ai_native_company` là bộ não và sơ đồ tổ chức (định nghĩa worker), còn repo này là hệ thống liên lạc nội bộ cộng ca trực, nơi con người điều phối đội mỗi ngày.

Ảnh xạ Worker thành phiên

Trong thực tế vận hành, mỗi AI Worker triển khai thành một phiên Claude:

AI Worker (vai trò)	Phiên relay	Topic Telegram	Lịch
Worker Marketing	marketing-daily	topic riêng	standup 9h mỗi ngày
Worker Sales	sales-pipeline	topic riêng	tổng hợp 18h mỗi ngày
Worker Finance	finance-report	topic riêng	báo cáo 0h ngày 1 hàng tháng
Worker Research	research-fleet	nhiều topic	broadcast khi cần

Mỗi worker làm việc trong thư mục riêng, có ngữ cảnh riêng, báo cáo vào topic riêng. Bạn mở Telegram và thấy cả đội đang làm gì.

Một ngày vận hành điển hình

Sáng: mở Telegram, các worker đã tự chạy theo lịch, đọc báo cáo trong từng topic. Trong ngày: giao việc cụ thể bằng cách nhắn trong topic tương ứng, nhận phản hồi theo thời gian thực. Chiều: broadcast một chỉ thị chung cho cả đội (ví dụ “ưu tiên dự án X”). Tối: kiểm toán, xử lý cảnh báo, lên lịch cho ngày mai.

Toàn bộ diễn ra trên điện thoại. Không cần mở terminal, không cần ngồi trước máy. Đó là lời hứa của AI Native Company: một người, một đội, vận hành từ chiếc điện thoại trong tay.

Phụ lục A: Bảng quy tắc

Hai mươi bảy quy tắc (R-00 đến R-27) là bài học gỡ rối thực tế, đúc từ các lỗi đã gặp. Hiếu chúng giúp tránh lặp sai lầm.

Mã	Quy tắc
R-00	Đọc JSON bằng công cụ Read; python3 chỉ để ghi/chéo tham chiếu
R-01	Xác nhận trước khi xóa phiên (nút Telegram)
R-01b	Chuyển thư mục giữ nguyên topic_id
R-02	Tên phiên: chữ thường gạch ngang, duy nhất
R-03	Mọi phiên có một topic Telegram
R-04	launchd là bộ lập lịch chính; CronCreate chỉ cho tạm trong phiên
R-05	Mọi agent dùng nhân <code>com.vccc.<tên></code>
R-06	plist nằm trong <code>~/Library/LaunchAgents/</code>
R-07	Broadcast tối đa mười phiên
R-08	Khoảng giám sát tối thiểu mười phút
R-09	Tự khởi động lại tối đa ba lần/giờ, rồi báo con người
R-10	Thoát dấu nháy đơn trong prompt send-keys
R-11	Kiểm tra phiên sống trước khi gửi
R-12	Bốn nguồn dữ liệu phải đồng bộ khi thay đổi
R-13	cron_list phát hiện mờ côi cả hai hướng
R-14	launchd tự ghi stdout/stderr (không cần tee)
R-15	cron_delete dỡ launchctl trước khi xóa plist
R-16	cron_update = dỡ → sửa → nạp lại (nguyên tử)
R-17	Tên tác vụ không chứa dấu chấm
R-18	Luôn khởi động Claude qua <code><CCC_BIN> run</code>
R-19	Script cầu nối đặt PATH ngay sau <code>set -euo pipefail</code>
R-20	Hàm định nghĩa trước lần gọi đầu tiên
R-21	Script cầu nối kiểm tra phụ thuộc đầu
R-22	Chạy kiểm toán sau mỗi lần tạo/sửa cron
R-23	Chuyển thư mục là tạo lại, không chỉ sửa cấu hình
R-24	Không bao giờ chuyển múi giờ: giờ cron =

Mã	Quy tắc
	giờ plist, giờ địa phương
R-25	Thư mục mặc định \$PROJECTS_DIR/<tên>, trích dẫn nếu có khoảng trắng
R-26	Thư mục mới phải có .claude/settings.json bỏ quyền trước khi <code>ccc run</code>
R-27	Đổi tên cửa sổ tmux về tên phiên sau <code>ccc run</code>

 **ĐIỂM CHÍNH:** R-19, R-20, R-21 là bộ ba về “PATH và thứ tự”, bảo script cầu nối chạy đúng trong môi trường launchd tối giản. R-24 (không chuyển múi giờ) và R-26 (bỏ quyền) là hai bug khó tìm nhất nếu vi phạm.

Phụ lục B: Khắc phục sự cố

Bot không nhận tin nhắn vào. Chạy `ccc doctor`. Kiểm tra dịch vụ lắng nghe đang chạy, thử `ccc listen` thủ công. Xác nhận bot là quản trị viên trong nhóm, và nhóm đã bật Topics.

Tác vụ cron không kích hoạt. Chạy `relay_cron_list`, nhìn hai cột “loaded” và “disk”. Plist có trên đĩa nhưng chưa nạp thì chạy `launchctl load ~/Library/LaunchAgents/com.vccc.<tên>.plist`. Xem lỗi bằng `tail ~/.claude/ccc-orchestrator/logs/<tên>.err`.

Tác vụ chạy sai giờ (trễ hoặc lệch vài tiếng). Bạn đã chuyển múi giờ nhầm. launchd dùng **giờ địa phương**, không phải UTC (R-24). Cron ghi 0 8 (tám giờ sáng) thì plist phải là Hour=8, không phải Hour=1.

Claude hỏi quyền liên tục qua Telegram. Thư mục dự án thiếu file bỏ quyền. Chạy `mkdir -p <dir>/.claude && echo '{"permissionMode":"bypassPermissions"}' > <dir>/.claude/settings.json` trước khi `ccc run` (R-26).

Script cầu nối báo “command not found” ở dòng lạ. Môi trường launchd có PATH tối giản. Đảm bảo script đặt `export PATH=...` ngay sau `set -euo pipefail` (R-19) và có khối kiểm tra phụ thuộc đầu (R-21).

Hàm không chạy, hoặc sụp đổ âm thầm. Hàm được gọi trước khi định nghĩa (R-20). Di chuyển mọi định nghĩa hàm lên trên mọi điểm gọi trong script cầu nối.

Cửa sổ tmux bị đổi tên sau khi tạo. `ccc run` có thể tự đổi tên cửa sổ. Force đổi lại: `tmux rename-window -t <id> <tên>` (R-27).

Lỗi “session already exists” mà không thấy phiên. Có entry mờ côi trong cấu hình ccc. Dọn bằng tay: xóa entry đó trong `~/.config/ccc/config.json`, rồi chạy `relay_list` xác nhận.

Phụ lục C: Thuật ngữ

Relay: phần mềm đứng giữa Claude Code và Telegram, chuyển tin nhắn hai chiều.

Session (phiên): một Claude Code chạy trong một cửa sổ tmux, gắn với một topic và một thư mục dự án.

Topic: một kênh trong nhóm Telegram forum; mỗi phiên map một-một với một topic.

group_id: số định danh một nhóm Telegram (dạng -100xxxxxxxxxx cho supergroup).

BotFather: bot chính thức của Telegram để tạo và quản lý bot khác.

launchd: bộ lập lịch native của macOS, thay thế cron. Sống qua khởi động lại và thức giấc từ chế độ ngủ.

plist (danh sách thuộc tính): file XML định nghĩa một tác vụ launchd.

cron: biểu thức năm trường (phút giờ ngày tháng thứ) mô tả lịch chạy.

registry.json: file lưu siêu dữ liệu điều phối (mục đích, thẻ, trạng thái) cho mỗi phiên.

schedules.json: file lưu các tác vụ định kỳ (cron, prompt, lịch sử).

broadcast: gửi cùng một lệnh cho nhiều phiên cùng lúc.

ccc (Claude Code Companion): engine relay của kidandcat, phần nền móng mà repo này xây trên.

AI Worker: nhân sự số chuyên biệt trong mô hình AI Native Company.

Cuốn sách này thuộc hệ sinh thái [AI Native Company](#). Repo và mã nguồn: [claude-telegram-multi-relay](#). Engine nền móng: [ccc \(kidandcat\)](#).